

Extending the SDD Approach Beyond Spotter

Spec-Driven Development: building, porting and recreating open-source and published projects

Treat the source as a reference design, not a build procedure.

An existing repository or published article is the **schematic and parts list**. The specification captures it; the AI agent does the **layout and detailed BOM** for your platform. The amateur radio enthusiast supplies judgment at the gaps, not build expertise. This is an *assistant* for amateur radio project makers, not a route to a 100%-finished project.

The four-step framework

1 Extract	2 Constrain	3 Surface gaps	4 Generate & observe
The agent reads the repo or article and drafts the Project Brief : behavior, inputs, outputs, interfaces. No build mechanics.	Architecture names the target (Mac, Arduino variant) and a one-command toolchain; Ground Rules ban Make and scripts.	The agent produces an explicit open-questions list . Answer what you can; the rest stays marked TBD.	Acceptance is behavior a typical radio operator can see: display shows X, LED blinks at 1 Hz, known input gives known output.

Questions

Can an experimenter build an open-source repo without fighting Make, scripts and dependencies?

- Yes. The Ground Rules ban custom build scripts; the Architecture declares and pins dependencies (pyproject with uv, or platformio.ini). One command to run.
- You never resolve transitive dependencies by hand.

Can I port a Windows-only repo to Mac, or from any platform A to B?

- **Regenerate, don't port.** The repo is the reference design: the agent extracts its behavior into the Project Brief, and the Architecture names the new target.
- You never need to have built a binary on the original platform.
- The result is a functional equivalent, a fork by regeneration rather than a line-by-line port. It is the same approach planned for Spotter's iPhone version.

Arduino apps from scratch, or ported to another Arduino variant or microcontroller family (Espressif ESP32, Microchip PIC)?

- From scratch: the same as Spotter, an authored masterplan with a new target.
- Porting: the Project Brief names *functions*; the Architecture maps pins, peripherals, core and libraries for the specific target board or chip. Pin the core and library versions.

What about a published project with prose only and no code?

- The article is a schematic and parts list without a layout. It is often **more** determinate than a typical first SDD spec, because the author already built and debugged it.
- Expect to iterate by hand on the gaps. Build from your derived spec, not the article text.

A radio enthusiast can't judge whether a spec is complete. So what?

- Nobody can judge a first spec's completeness, enthusiast or professional developer.
- SDD's contribution is making incompleteness **visible**: the gaps list is an artifact, not a surprise at run time.

How does a project builder verify that it works?

- Write observable acceptance criteria into the spec up front. If you can't see a behavior, spec a test hook that makes it visible (serial print, status LED).

Does this produce a finished product?

- No. It gets you to functioning code you can iterate on.

What about licensing?

- **License follows the source.** Private use carries no GPL obligations.
- Publishing the regenerated repo, or handing someone firmware, is distribution: GPL in means GPL out, with source. Keep the original license and attribution.
- Ground Rules can require attribution, but it can't add a "personal use only" restriction to GPL-derived code. Not legal advice.

Watch-outs

- **Hidden behavior**, such as config defaults, registry keys, protocol quirks and timing, never reaches the extracted Project Brief.
- **Extracted bugs become requirements.** Separate intended from observed behavior.
- **No path upstream:** regenerated code can't merge upstream fixes.
- **Espressif and PIC libraries:** the agent may pick AVR-only libraries (direct registers, PROGMEM, timers) or invent APIs. Confirm the core first.
- **Observable checks miss** rare timing and interrupt races. Fine for an assistant; know the limit.
- **Errata:** check the article for published corrections first, or a schematic error becomes a requirement.
- **Licensing:** sharing GPL-derived code or firmware is distribution. Check every library's license too; Arduino cores and libraries are often LGPL.
- **Toolchain install is still failure #1:** board-manager URL, USB drivers, the Espressif BOOT button for flashing, and a PICkit programmer with MPLAB X for PIC.

Remember...

- The spec is the schematic; the agent does the layout.
- SDD doesn't tell you what's missing. It makes what's missing visible.
- License follows the source.